

Student : Krish Gupta

Beginner foundations and simulator selection • 4 - 17 September

Task 1 : Goal, Deliverables & Current Status

Goal

Become comfortable with Python, NumPy, OpenCV, Linux, Git and ROS 2 concepts, then select a simulator using evidence.

Planned deliverables

Setup log + Git repo → three small exercises → simulator comparison → recommendation + risks → 5-minute vision / ROS 2 demo.

Current status

Core technical work completed. Remaining acceptance-gate items: mentor verification on test images and mentor approval of the selected simulator.

Evidence collected

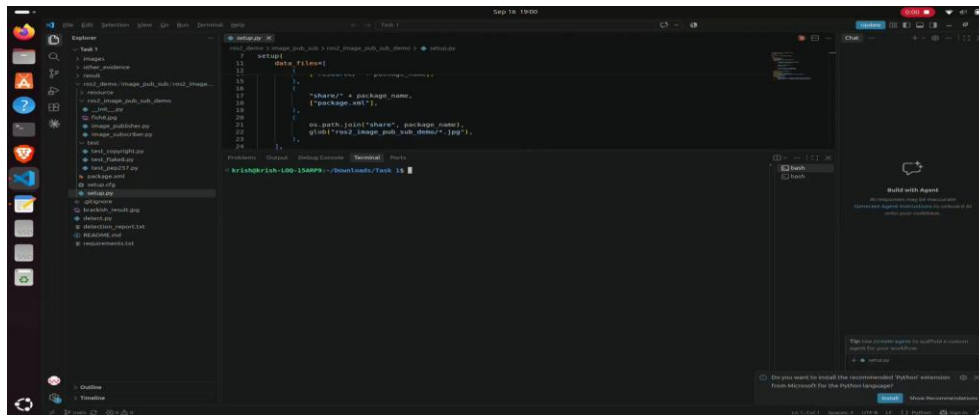
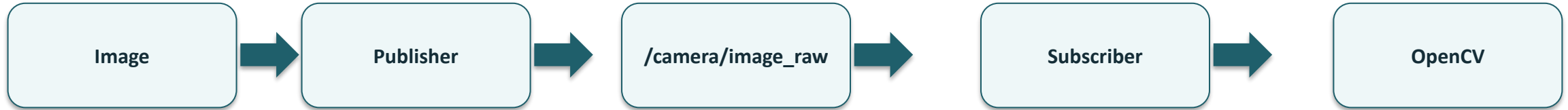
OpenCV results → Brackish YOLO result → ROS 2 package/topic evidence → Gazebo world/robot → Stonefish build → Unity assessment

Next checkpoint

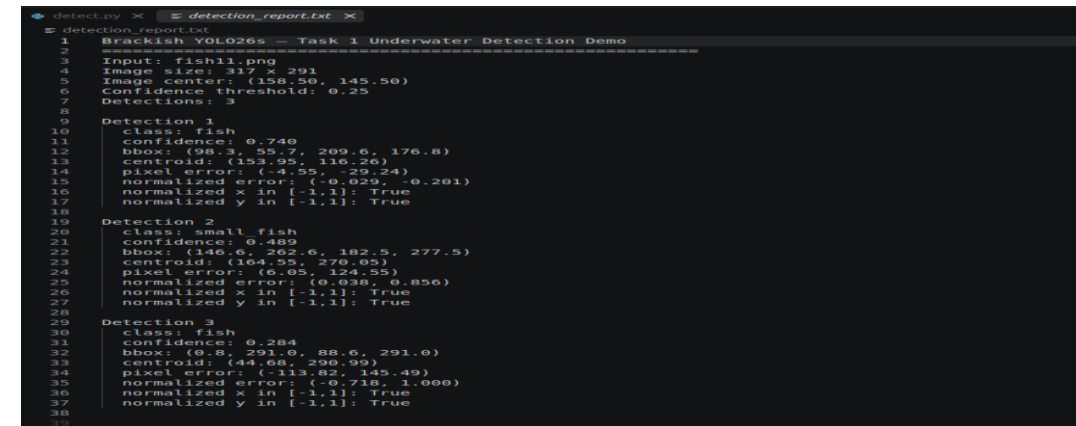
Improve YOLO Detection models and train them on certain data → Create a stable scene in the confirmed simulator that can be launched and manually controlled through ROS2

ROS 2 Publisher / Subscriber + OpenCV Image Topic

Actual package evidence and the intended message flow



Evidence: ROS 2 package with publisher/subscriber entry points



Evidence: ROS 2 image topic / sensor_msgs/msg/Image

Publisher

Reads image with OpenCV and converts it to sensor_msgs/msg/Image using CvBridge.

Subscriber

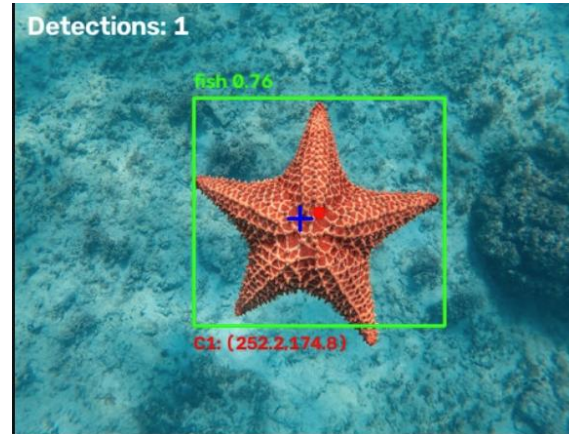
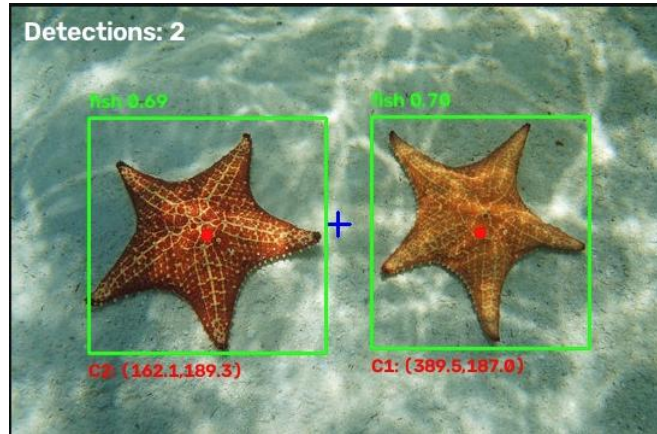
Receives /camera/image_raw, converts the ROS image back to an OpenCV image with CvBridge, and displays/processes it.

Topic viewing

rqt_image_view is used separately to visualize /camera/image_raw and demonstrate the image topic.

Underwater Vision -- Brackish YOLO26s & Centroid Output

Result image, measured values, and current failure cases



RESULT: Brackish YOLO26s detection on underwater image

OpenCV fundamentals

Image dimensions, image centre, pixel error and normalized error. The simple contour method was kept as a beginner exercise.

Underwater vision

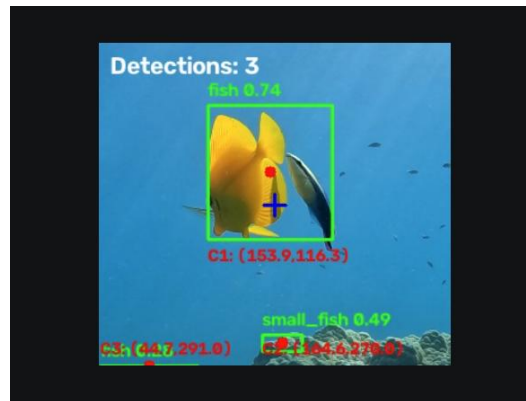
Brackish YOLO26s was tested on underwater images. Detection output provides target locations from bounding boxes; centroid is calculated from detected objects

ROS 2 fundamentals

Publisher → /camera/image_raw → subscriber using sensor_msgs/msg/Image and CvBridge to convert between ROS images and OpenCV.

Current limitation

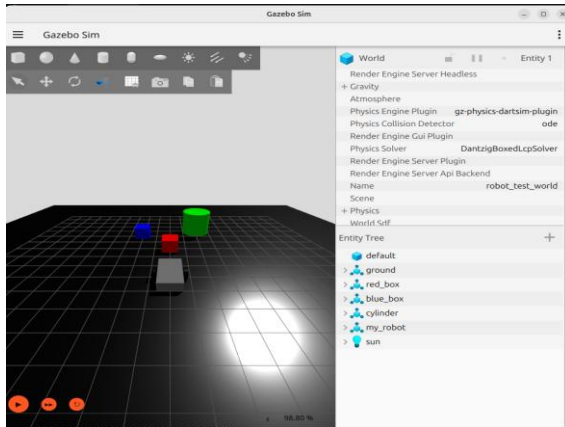
Brackish YOLO26s works on clear tested targets, but we are still unable to reliably detect objects in noisy, low-contrast, occluded or unclear images. This remains a validation risk.



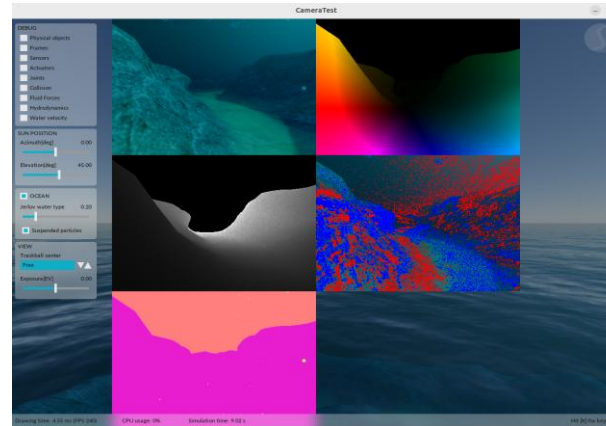
FAILED RESULT: simple contour approach

Simulator Comparison -- Gazebo vs Stonefish vs Unity

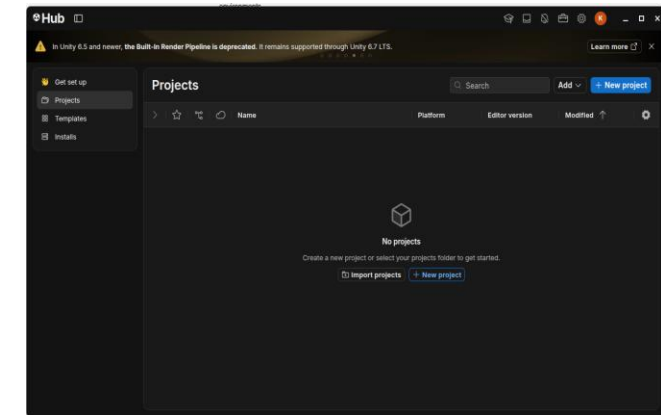
Evidence from hands-on testing plus documented assessment



Gazebo Harmonic — custom world / robot

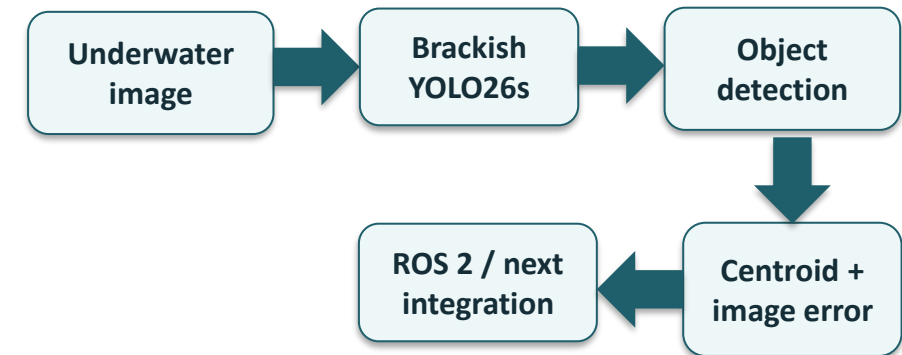


Stonefish — successful source build



Unity Hub — setup assessment

Criterion	Gazebo Harmonic	Stonefish	Unity
ROS 2 fit	Strong ros_gz bridge	Strong ROS 2 wrapper	Good ROS Connector
Marine / underwater physics	General robotics; less specialized	Marine-focused hydrodynamics	General-purpose; custom implementation
Camera / sensors	Camera + sensor ecosystem	Underwater rendering + sensors	Strong visual/sensor tooling
Hands-on status	Installed + world + robot tested	Built from source + tests run	Hub assessed; Editor not installed
Beginner setup / compute	Good on current laptop	Good after build; GPU available	Editor footprint/setup heavier
Best evidence from this task	ROS 2 integration + robot world	Underwater specialization + successful build	Viable visual simulation option



Main Blockers, Next Two-Week Task & Expected Result

Close Task 1 cleanly and transition into controlled underwater simulation

Main blockers / failed attempts

Simple contour detection can fail on underwater backgrounds. Brackish YOLO26s still has unresolved failures on noisy or unclear targets. A single 2D centroid cannot recover unrestricted 3D position without depth/range information.

Next two-week task

Build the minimum underwater robotics scenario: vehicle + camera/sensors + ROS 2 interfaces + repeatable test world.

First action

Get mentor approval on the simulator, then create the smallest Stonefish/Gazebo test scenario and define the ROS 2 interfaces before adding complexity.

Expected result

A repeatable simulation where sensor data can be inspected through ROS 2 and the vision pipeline can be evaluated against known target positions.

Acceptance checks

Centroid values pass mentor test images → normalized errors remain within $[-1,1]$ → publisher/subscriber + image topic are demonstrable → simulator recommendation approved.